BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. $0.1000_2 \times 2^3$.

Q2. 11111111.

Q3. 8.

Q4. 0.1100_2 = decimal 0.75.

You Do pages · Independent questions

Q1. So corresponding mantissa bits represent the same place values.

Q2. Align B to 0.0100×2^3. Subtract $0.1100 - 0.0100 = 0.1000 \times 2^3 = 4$.

Q3. Right-shifting may discard significant low-order bits. They cannot all be held in the available mantissa, so rounding or truncation can produce an inexact result.

Exit Ticket · Model answer

True mathematically; finite-width storage may discard significant bits.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 195 · Floating-point arithmetic and precision

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Normalise positive $0.0100_2 \times 2^4$.

Q2. What is −1 in 8-bit two's complement?

Q3. Calculate 0.5×2^4.

Q4. Calculate binary $0.1000_2 + 0.0100_2$.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

exponent alignment — Express operands using the same power of two.

rounding — Choose a nearby representable value for an exact result.

truncation — Discard bits beyond the available precision.

overflow — Numerical result exceeds the representable range.

precision loss — Significant binary information cannot be stored.

New learning

Align exponents — Before adding mantissas, both terms need the same power of two. Shift B right one place when raising its exponent by one.

Add the mantissas — $0.1000_2 + 0.0100_2 = 0.1100_2$; keep the common exponent 3.

Check in decimal — $0.1100_2 \times 2^3 = 0.75 \times 8 = 6$, matching 4 + 2.

Worked example

Calculate $0.1100_2 \times 2^3$ minus $0.1000_2 \times 2^2$.

1. Align — B=0.1000×2^2 becomes 0.0100×2^3.

2. Subtract — $0.1100 - 0.0100 = 0.1000$.

3. Check — $0.1000_2 \times 2^3 = 4$; this matches decimal 6 − 2.

We Do · Practise with guidance

Add $0.1010_2 \times 2^3$ and $0.1000_2 \times 2^1$.

```
A = 0.1010 × 2³ = 5
B = 0.1000 × 2¹ = 1
Aligned B = ?
Result = ?
```

Align

Raise B's exponent 1→3 by shifting its mantissa right twice: 0.1000 becomes 0.0010.

Add and check

0.1010 + 0.0010 = 0.1100×2^3 = decimal 6.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Why must exponents be aligned before binary floating-point addition?

Question 2

Calculate $0.1100 \times 2^3 - 0.1000 \times 2^2$ with working.

Question 3

Explain how exponent alignment can lose precision when the mantissa has a fixed number of bits.

Exit Ticket · Answer before checking

When increasing an exponent by 1 during alignment, the mantissa must shift right 1 place to preserve its value.